

Activity 1: Refactor the EMI Calculator using AI

Purpose

Learn how to use AI to analyze existing code, identify areas for improvement, and refactor the application while preserving its functionality.

Pre-requisites

- Completion of Day 2 activities
- Working Oracle JET EMI Calculator
- VS Code with Cline configured

Scenario

The EMI Calculator has been successfully implemented and is working as expected. During the code review meeting, the Solution Architect observed that although the application functions correctly, the code can be improved for better readability, maintainability, and long-term support.

Your task is to use AI to analyze the existing implementation, identify refactoring opportunities, and improve the code without changing the application's behavior.

Tasks

Task 1: Analyze the Existing Code

Before making any changes, analyze the application and identify areas that can be improved.

Suggested Prompt

Analyze the current Oracle JET and KnockoutJS implementation of the EMI Calculator.

Before suggesting any code changes:

- Summarize the purpose of the application.
- Identify code smells or design issues, if any.
- Highlight duplicate or repetitive code.
- Identify overly complex functions.
- Review variable and function naming.
- Suggest opportunities to improve readability and maintainability.

Explain your findings first.

Do not modify the code.

Task 2: Refactor the Application

Refactor the application based on the findings.

Suggested Prompt

Refactor the current EMI Calculator based on the identified improvements.

Requirements:

- Preserve all existing functionality.
- Modify only the required code.
- Improve readability and maintainability.
- Simplify the implementation wherever possible.
- Improve variable and function names if required.
- Remove duplicate logic.
- Follow Oracle JET and KnockoutJS best practices.

Before generating code, explain what will be changed.

After refactoring, summarize all improvements.

Task 3: Compare the Original and Refactored Code

Review the improvements made during refactoring.

Suggested Prompt

Compare the original implementation with the refactored version.

For each improvement, explain:

- What was changed.
- Why the change was required.
- How it improves code quality.
- Whether the application's functionality remains unchanged.

Present the comparison in a table.

Task 4: Validate the Refactored Code

Perform a final review to ensure the refactoring did not introduce any issues.

Suggested Prompt

Review the refactored Oracle JET application.

Verify that:

- The original functionality is preserved.
- No new issues have been introduced.
- The code follows Oracle JET and KnockoutJS best practices.
- The application is easier to understand and maintain.

Suggest any additional improvements if required.

Do not modify the code.

Expected Deliverables

By the end of this activity, you should have:

- Analysis report highlighting refactoring opportunities.
- Refactored EMI Calculator code.
- Comparison between the original and refactored implementation.
- Validation report confirming the functionality is unchanged.